

Pipelining and Vector Processing

Prepared By Er Abhishek Singh
Computer Engineer
Department Of IT
Government of Nepal

Chapter #6- Pipeline and Vector Processing

- Pipelining
- Parallel Processing
- Arithmetic Pipeline
- Instruction Pipeline
- RISC Pipeline
- Vector Processing
- Array Processing

Pipelining

- Motto: Do not let any component ideal (but try to optimize them)
- How???
- A process is divided in to several sub-processes where each sub-process is associated with a special dedicated segment of the process
- The overlapping of computation is made possible by associating a register with each segment in the pipeline
- The registers provide isolation between each segment so that each can operate on distinct data independently (*in most cases simultaneously*)
- The simplest way of viewing the pipeline structure is to imagine that each segment consists of an input register followed by a combinational circuit
 - The register holds the data
 - The combinational circuit performs the sub-process in the particular segment

Illustration Example

$$P = A_i * B_i + C_i \text{ for } i = 1, 2, 3, \dots, 7$$

Subprocess #1: $R_1 \leftarrow A_i, R_2 \leftarrow B_i$

(Input A_i and B_i)

Subprocess #2: $R_3 \leftarrow R_1 * R_2, R_4 \leftarrow C_i$

(Multiply and Input C_i)

Subprocess #3: $R_5 \leftarrow R_3 + R_4$

(Add C_i to the Product)

Notes:

Each segment has one or two registers and a combinational circuit
The five registers are loaded here with new data at every clock pulse
(A clock is applied to all registers)

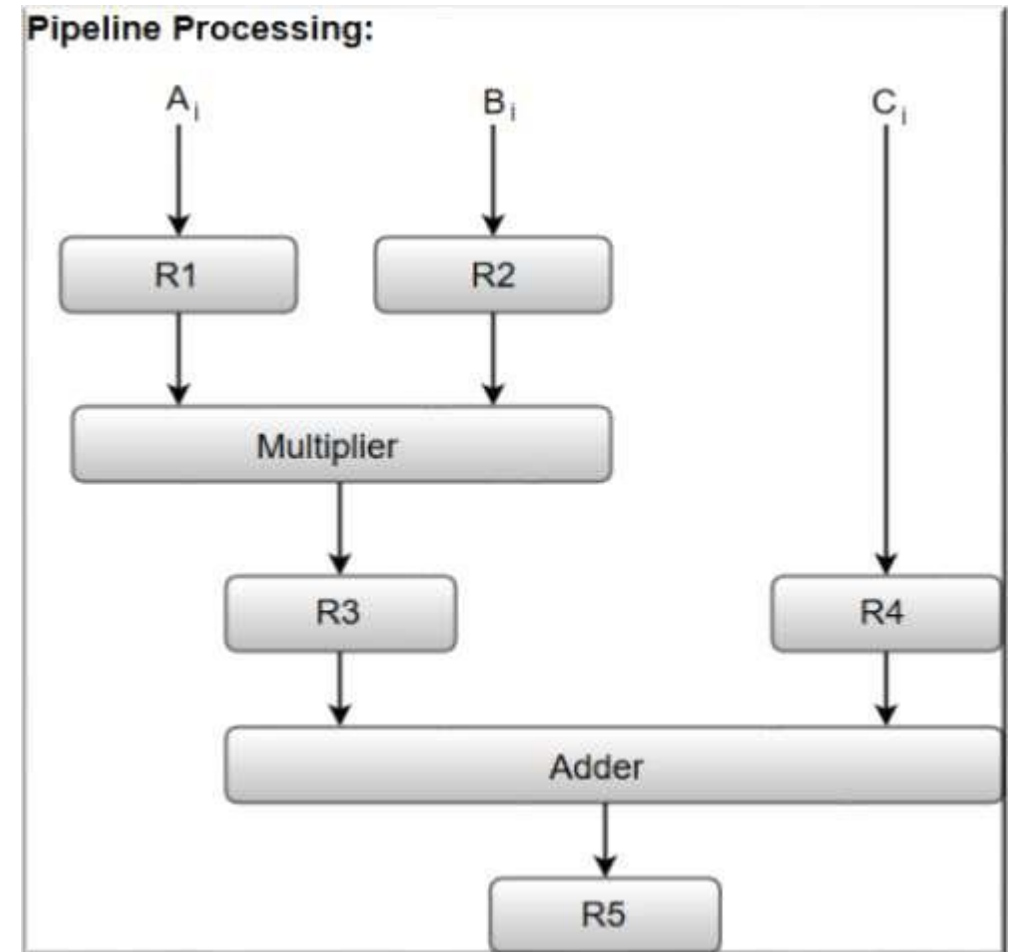
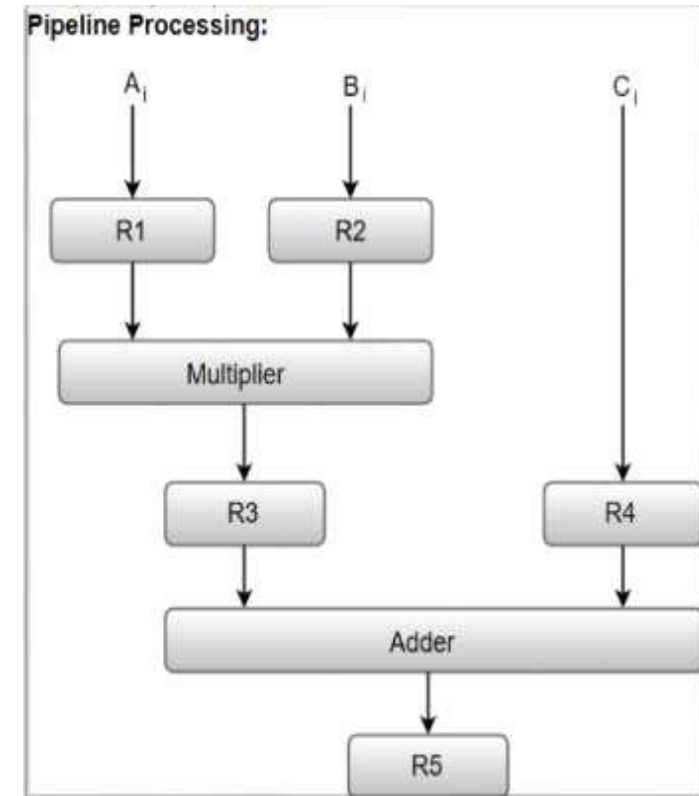
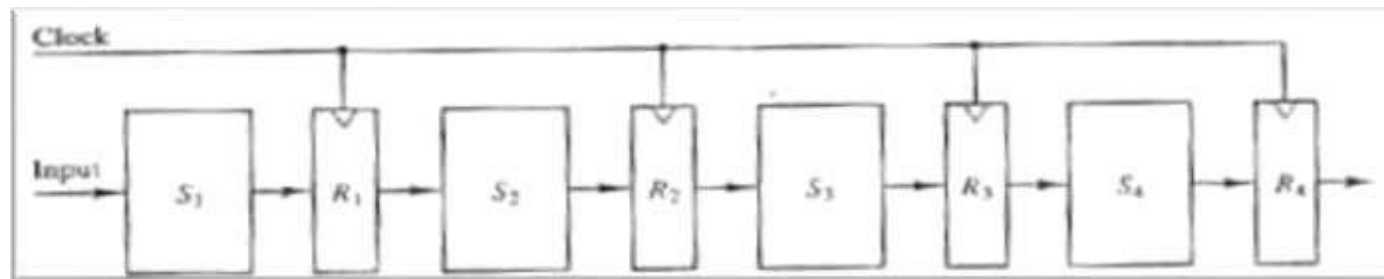


Illustration Example

- The five registers are loaded with new data at every clock pulse
- Table below shows the effect of each clock
- 3 Clock Cycles for 1 Output || 3 Subprocess * 7 = 21 Clock Cycles (via sequential)
- Via Pipeline: 9 Clock Cycles

Clock Pulse Number	Segment 1		Segment 2		Segment 3
	R1	R2	R3	R4	R5
1	A_1	B_1	--	--	--
2	A_2	B_2	$A_1 * B_1$	C_1	--
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	--	--	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	--	--	--	--	$A_7 * B_7 + C_7$

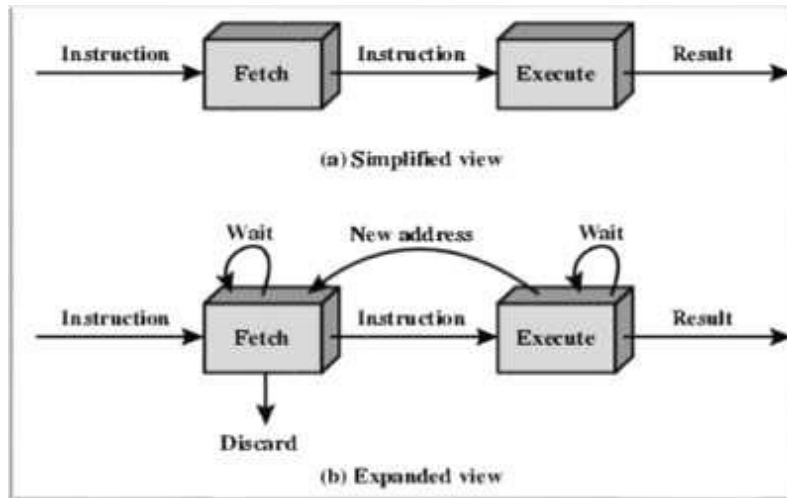


Six-Stage Pipeline (from Chapter #1)

(Assumed that each instruction goes through all six stages of the pipelines- this will not always be the case)

- Fetch Instruction (FI)
- Decode Instruction (DI)
- Calculate Operand (CO)
- Fetch Operand (FO)
- Execute Instruction (EI)
- Write Operand (WO)

(9 instructions from 54 to 14 times units)



		Time →													
		1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1		FI	DI	CO	FO	EI	WO								
Instruction 2			FI	DI	CO	FO	EI	WO							
Instruction 3				FI	DI	CO	FO	EI	WO						
Instruction 4					FI	DI	CO	FO	EI	WO					
Instruction 5						FI	DI	CO	FO	EI	WO				
Instruction 6							FI	DI	CO	FO	EI	WO			
Instruction 7								FI	DI	CO	FO	EI	WO		
Instruction 8									FI	DI	CO	FO	EI	WO	
Instruction 9										FI	DI	CO	FO	EI	WO

Pipeline Speedup Ratio

Mathematically, it can be given by

$$s = \frac{\text{time taken in non pipeline system}}{\text{time taken in pipeline system}} = \frac{nt_n}{(k+n-1)t_p}$$

Where,

k = no. of segments; n = no. of tasks; s = speed up ratio

a) For very large no. of tasks, k+n-1 tends to n, thus

$$s = \frac{t_n}{t_p}$$

b) If the time it takes to process a task is same in pipeline and non pipeline circuits, we will have,

$$t_n = kt_p$$

Thus,

$$s = \frac{kt_p}{t_p} = k$$

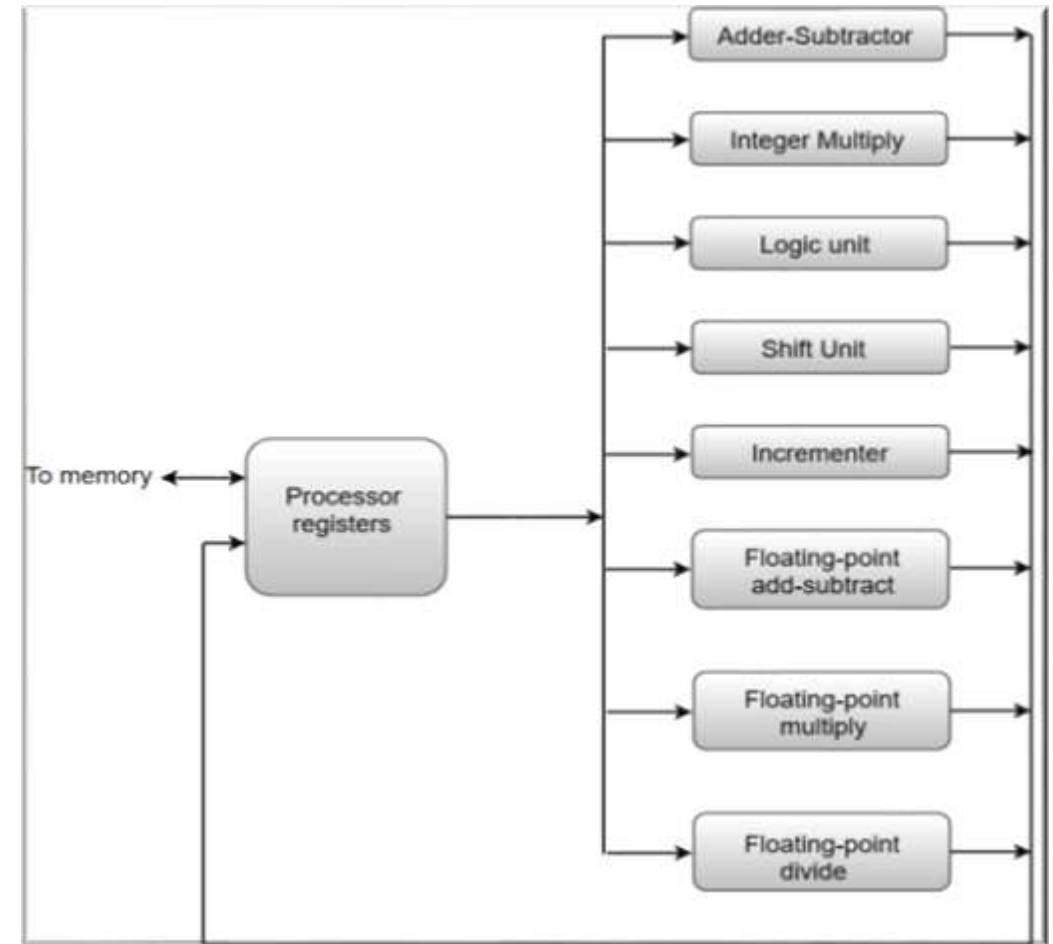
- This shows that the theoretical maximum speed up that a pipeline can provide is k, where k is the number of segments in the pipeline.

Parallel Processing

- In sequential processing, the CPU Fetches the instructions and Execute them in every Fetch-Execute cycle (one by one)
- i.e. once one instruction execution successfully completes the next instruction gets chance to enter in execution process and hence fetched (from main memory).
- The computer performance can be increased by using a fast computing approaches such as PARALLEL PROCESSING.
- Parallel processing announces a chance to execute multiple instructions simultaneously (i.e. concurrent execution of instructions)

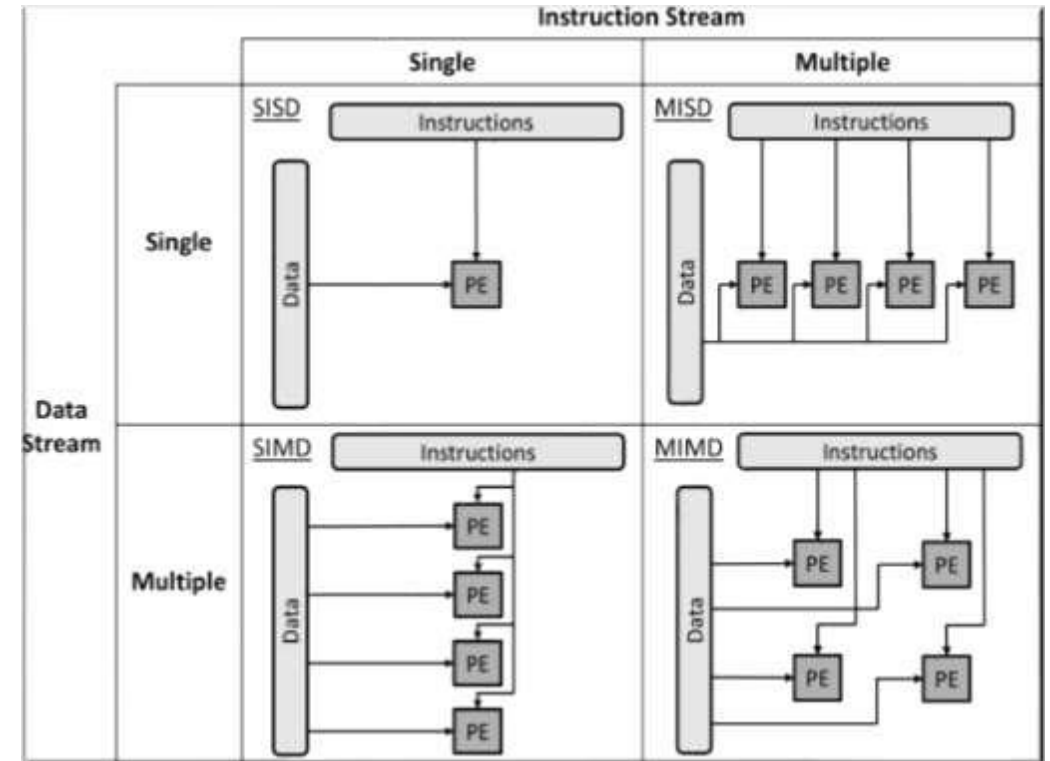
Parallel Processing

- Computer – {CPU, Memory, I/O}
- CPU- {**ALU**, CU, Register Array}
- ALU comes with several functional units
- aka Processor with Multiple Functional Units
- Functional units are independent units
- All the units can work concurrently
(Simultaneous Execution)



Flynn's Classification (Four Categories)

- Michael J. Flynn(1934, Stanford University- Professor)
- considers the organization of a computer system by:
 - the number of instructions (from **memory**) and
 - data items (in **processor** register)
- (that are manipulated simultaneously)
- Single instruction stream, single data stream (SISD)
- Single instruction stream, multiple data stream (SIMD)
- Multiple instruction stream, single data stream (MISD)
- Multiple instruction stream, multiple data stream (MIMD)
- [Under MIMD- We will study Pipeline Processing, Vector Processing, and Array Processing]



Single-Instruction, Single-Data (SISD) Systems

An SISD computing system is a uniprocessor machine which is capable of executing a single instruction, operating on a single data stream. In SISD, machine instructions are processed in a sequential manner, and computers adopting this model are popularly called sequential computers. Most conventional computers have SISD architecture. All the instructions and data to be processed have to be stored in primary memory. The speed of the processing element in the SISD model is limited(dependent) by the rate at which the computer can transfer information internally.

Single-Instruction, Multiple-Data (SIMD) Systems

An SIMD system is a multiprocessor machine capable of executing the same instruction on all the CPUs but operating on different data streams. Machines based on an SIMD model are well suited to scientific computing. So that the information can be passed to all the processing elements (PEs) organized data elements of vectors can be divided into multiple sets (N-sets for N PE systems) and each PE can process one data set

Multiple-Instruction, Single-Data (MISD) systems

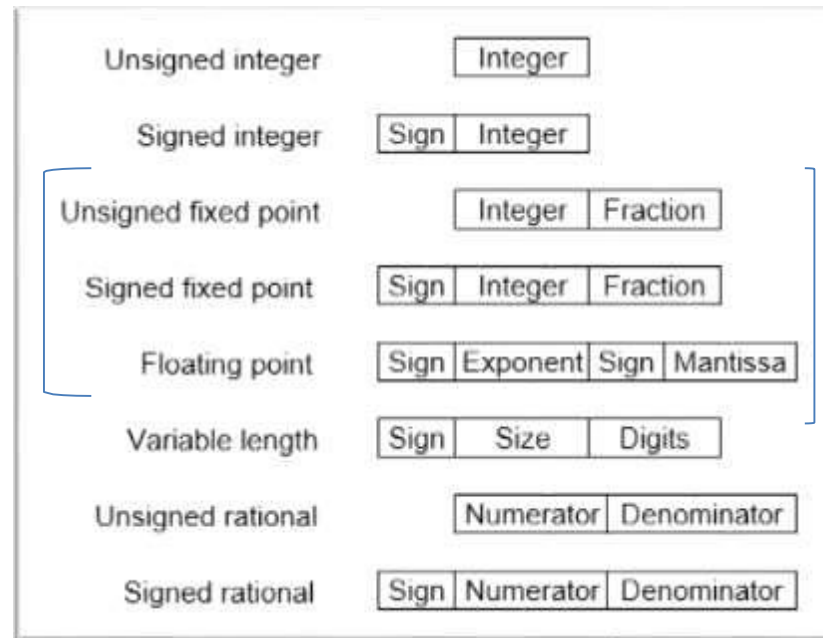
An MISD computing system is a multiprocessor machine capable of executing different instructions on different PEs but all of them operating on the same dataset. Example $Z = \sin(x) + \cos(x) + \tan(x)$ The system performs different operations on the same data set. Machines built using the MISD model are not useful in most of the application, a few machines are built, but none of them are available commercially.

Multiple-Instruction, Multiple-Data (MIMD) Systems

An MIMD system is a multiprocessor machine which is capable of executing multiple instructions on multiple data sets. Each PE in the MIMD model has separate instruction and data streams; therefore machines built using this model are capable to any kind of application. MIMD machines are classified into shared-memory and distributed-memory models depending on how processors connect to memory. In a shared-memory MIMD system (tightly coupled), all processors use the same global memory, and communication happens through it.

Arithmetic Pipeline

- Arithmetic pipeline is mainly useful to perform arithmetic operations
- Generally working with fixed point numbers and floating point numbers
- Floating Point Addition and Subtraction Operations

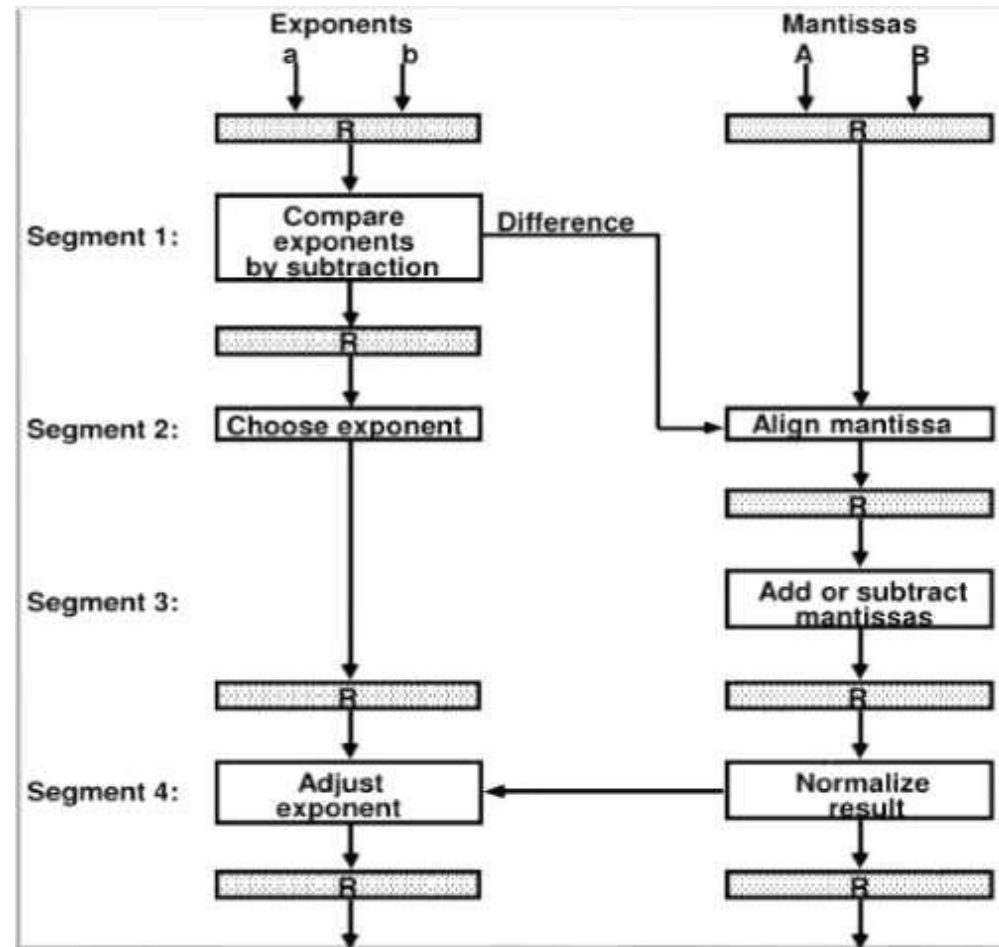


Arithmetic Pipeline (Typical Example)

- Floating-point adder/subtracter
- [1] Compare the exponents
- [2] Align the mantissa
- [3] Add/sub the mantissa
- [4] Normalize the result

$$X = A \times 10^2 = 0.9504 \times 10^3$$
$$Y = B \times 10^2 = 0.8200 \times 10^2$$

- 1) Compare exponents :
 $3 - 2 = 1$
- 2) Align mantissas
 $X = 0.9504 \times 10^3$
 $Y = 0.08200 \times 10^3$
- 3) Add mantissas
 $Z = 1.0324 \times 10^3$
- 4) Normalize result
 $Z = 0.10324 \times 10^4$

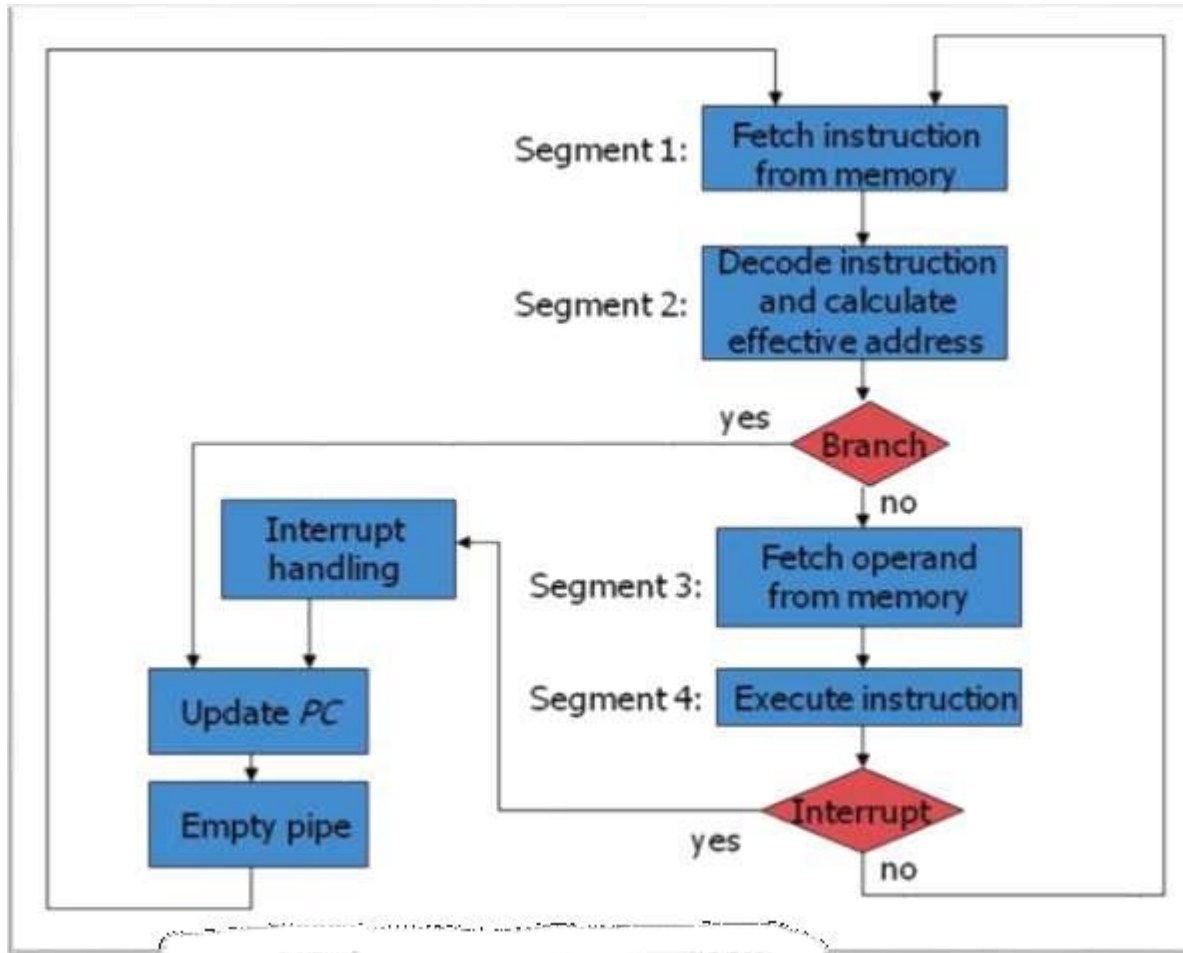


Instruction Pipeline

- Pipeline processing can occur not only in the data stream but in the instruction as well.
- Consider a computer with an **instruction fetch unit** and an **instruction execution unit** designed to provide a two-segment pipeline.
- Computers with complex instructions require other phases in addition to above phases to process an instruction completely.
- In the most general case, the computer needs to process each instruction with the following sequence of steps.
 - Fetch the instruction from memory
 - Decode the instruction
 - Calculate the effective address
 - Fetch the operands from memory
 - Execute the instruction
 - Store the result in the proper place

	Time →													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO	EI	WO					
Instruction 5					FI	DI	CO	FO	EI	WO				
Instruction 6						FI	DI	CO	FO	EI	WO			
Instruction 7							FI	DI	CO	FO	EI	WO		
Instruction 8								FI	DI	CO	FO	EI	WO	
Instruction 9									FI	DI	CO	FO	EI	WO

Instruction Pipeline Illustration



Step:	1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction:	FI	DA	FO	EX									
(Branch)		FI	DA	FO	EX								
			FI	DA	FO	EX							
				FI	—	—	FI	DA	FO	EX			
					—	—	—	FI	DA	FO	EX		
									FI	DA	FO	EX	
										FI	DA	FO	EX

Difficulty with Instruction Pipeline

- **Resource conflicts** caused by access to memory by two segments at the same time.
- **Data dependency** conflicts arise when an instruction depends on the result of a previous instruction, but this result is not yet available.
 - Solution: Use of an extra circuit {Hardware Interlocks, Operand Forwarding}
- **Branch difficulties** arise from branch and other instructions that change the value of PC.
 - Solution: Prefetch target instruction, Branch Target Buffer, Branch Prediction, Loop Buffer, Delayed Branch

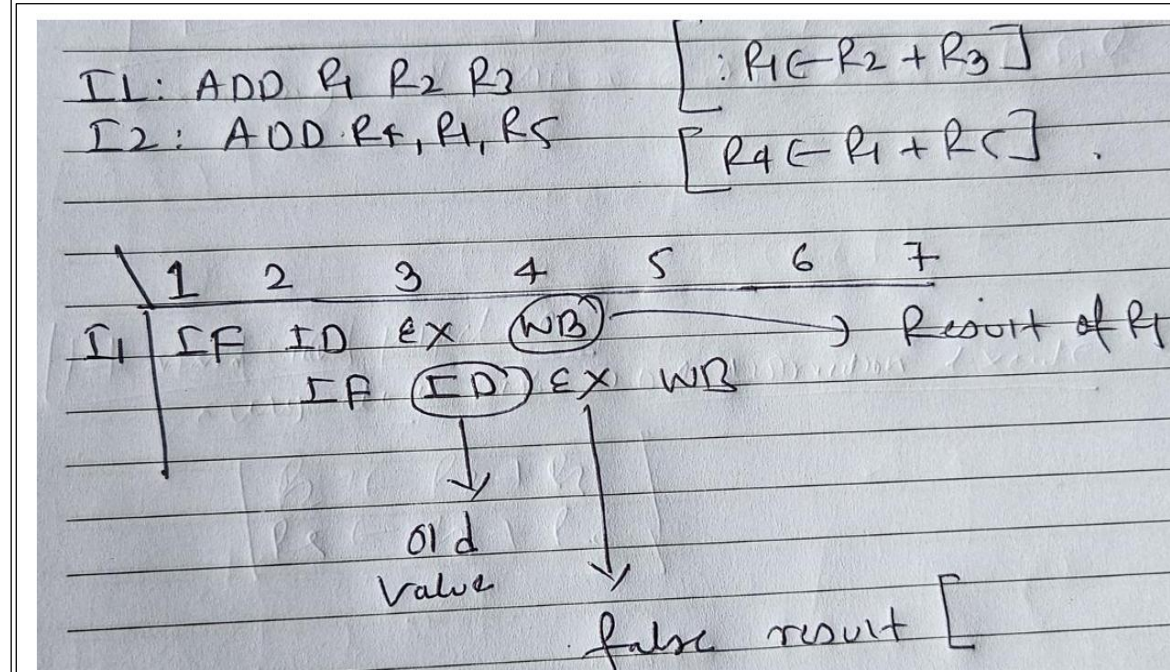
Data Conflict or Data Dependency:

- Data Conflicts occur when the pipeline causes an incorrect data value to be used.
- A data hazard occurs when instructions exhibit data dependencies such that one instruction depends on the result of a previous instruction that has not yet completed in the pipeline.
- Consider the following consecutive program statements.
- Data conflict occurs when multiple, inconsistent, or contradictory values exist for the same entity or data point across different systems, sources.

1: $R1 \leftarrow R2 + R3$

2: $R4 \leftarrow R1 + R3$

3: $R5 \leftarrow R6 + R3$



No-op insertion:

- The simplest is to have the compiler detect data Conflicts and insert no-ops to avoid the conflict.

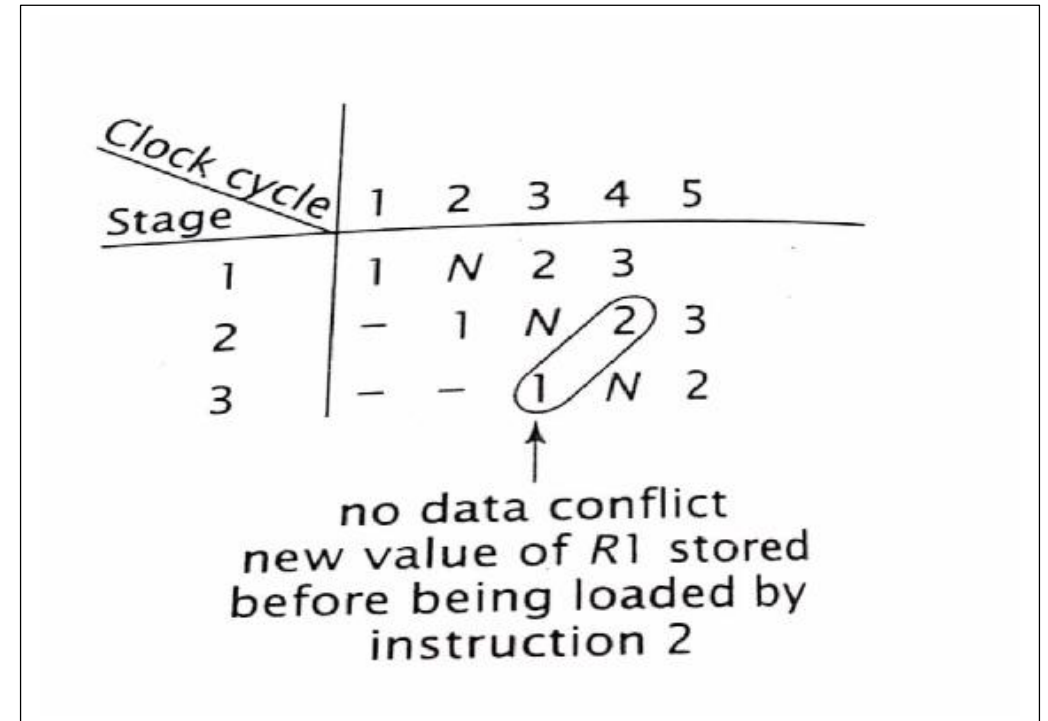
1: $R1 \leftarrow R2 + R3$

N: no-op

2: $R4 \leftarrow R1 + R3$

3: $R5 \leftarrow R6 + R3$

- The no-op statement in the second block delays the fetching of operands for the second Instruction by one clock cycle.
- This delay allows the last stage of the pipeline to Store the new value of R1 before it is loaded for use in executing the second instruction, thus avoiding data conflict.
- It reduces the overall system performance.
- The no-op instruction does not perform useful Work and require an extra clock cycle.



Instruction Re-Ordering:

- For some programs, the compiler can re order some Of the instruction store move the data conflict.

1: $R1 \leftarrow R2 + R3$

3: $R5 \leftarrow R6 + R3$

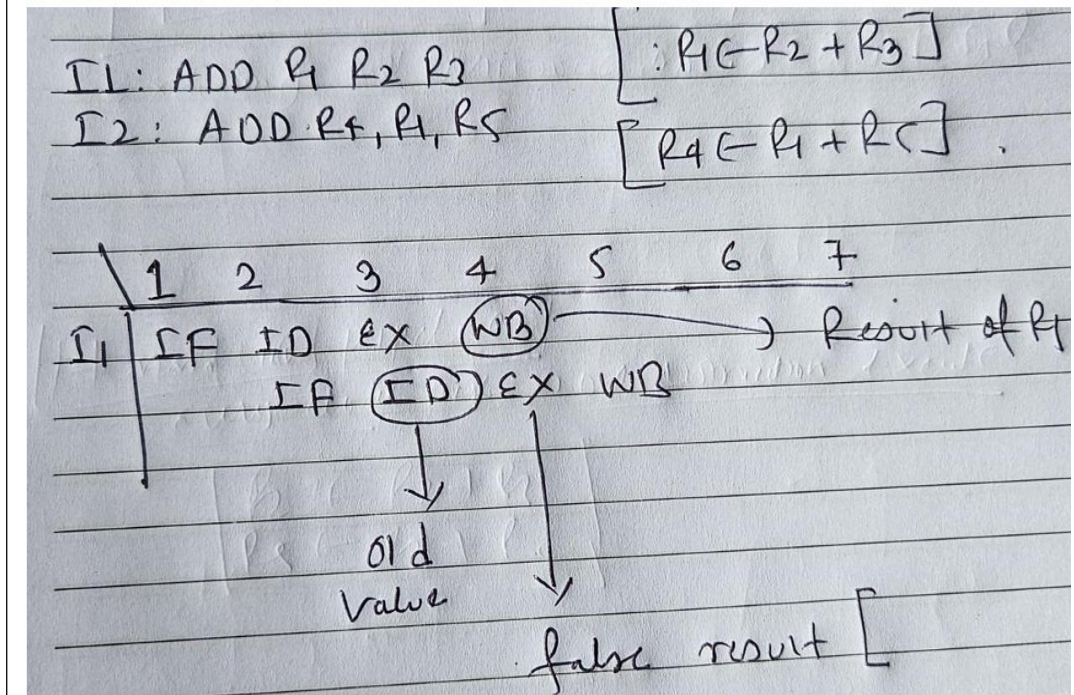
2: $R4 \leftarrow R1 + R3$

- Instruction reordering resolves the data conflict by Introducing a delay between the conflicting instructions.

- Unlike no-op insertion, instruction reordering Performs useful work during this time.

- It is not always possible to reorder the instructions Of a program to avoid data conflicts.

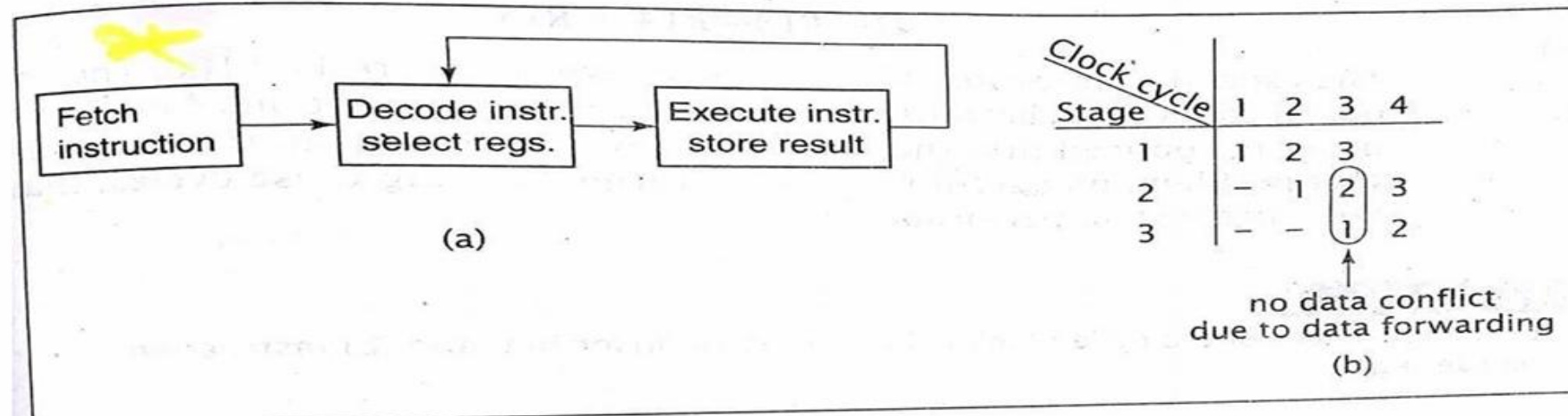
- No-op insertion and instruction reordering resolve Data conflicts using only the compiler.



Data forwarding:

- Hardware solution to the data conflict problem.
- After the instruction is executed, its result is stored just as before, but the result is also forwarded directly to the stage that selects registers (retrieves operands).
- That stage gets the new value of the operand directly from the execute instruction stage of the pipeline before (or at the same time as) it is stored in the appropriate register

Data forwarding: (a) modified three-stage pipeline and (b) execution trace of the code block



Branch conflicts or Control Hazards:

The following code segment illustrates the branch conflict problem:

1:R1←R2+R3

2:R4←R5+R6

3:JUMP10

4:R7←R8+R9

5:R10←R11+R12

:::

10:R13←R14+R15

Clock cycle	1	2	3	4	5	6
Stage	1	2	3	4	5	6
1	1	2	3	4	5	10
2	-	1	2	3	4	5 10
3	-	-	1	2	3	4 5

↑
these instructions
should not be executed

Fig: execution trace of the code block illustrating a branch conflict

After instruction 3 is executed, the CPU should branch to instruction 10; however, instructions 4 and 5 are already in the pipeline before instruction 3 is executed.

Solution to Branch Conflicts Problem (1)

No-op insertion:

```
1:    R1 ← R2 + R3
2:    R4 ← R5 + R6
3:    JUMP 10
N1:   no-op
N2:   no-op
4:    R7 ← R8 + R9
5:    R10 ← R11 + R12
      :
10:   R13 ← R14 + R15
```

The no-ops introduce a delay sufficient to ensure that instructions 4 and 5 are never introduced into the pipeline.

Instruction reordering:

```
3:    JUMP 10
1:    R1 ← R2 + R3
2:    R4 ← R5 + R6
4:    R7 ← R8 + R9
5:    R10 ← R11 + R12
      :
10:   R13 ← R14 + R15
```

Clock cycle Stage	1	2	3	4	5	6
1	1	2	3	N1	N2	10
2	-	1	2	3	N1	N2 10
3	-	-	1	2	3	N1

(a)

Clock cycle Stage	1	2	3	4	5
1	3	1	2	10	
2	-	3	1		
3	-	-	3		

(b)

Fig: execution traces of the code block using a) no op insertion b) Instruction reordering

Stall insertion can also be used to handle branch conflicts.

- When the pipeline recognized a branch instruction, it inserts stalls into the pipeline to delay the fetching of next instruction until after the branch instruction had been completed.

Annulling (1):

- In annulling, the instructions proceed through the pipeline as they normally would.
- If an instruction should not have been executed, because a previous instruction branched away from it, its result are not stored.
- Even though it might have been executed, as long as no results are stored, it is as if the instruction was never processed.

```
1:      R10 ← 10
2:      R1 ← R1 + R3
3:      R2 ← R2 + R3
4:      R10 ← R10 - 1
5:      IF(R10 ≠ 0) THEN GOTO 2
6:      R4 ← R5 + R6
7:      R7 ← R8 + R9
```

Annulling (2):

- The statements 5, 6, and 7 are all in the pipeline during clock cycle 7, even though statement 5 should be followed by statement 2.
- The execution of statements 6 and 7 are annulled by the pipeline hardware, which knows that the branch in statement 5 is taken.
- During clock cycle 61, the branch is not taken and the loop terminates.
- This time, the execution of statements 6 and 7 during the following two clock cycles are not annulled and their results are stored.

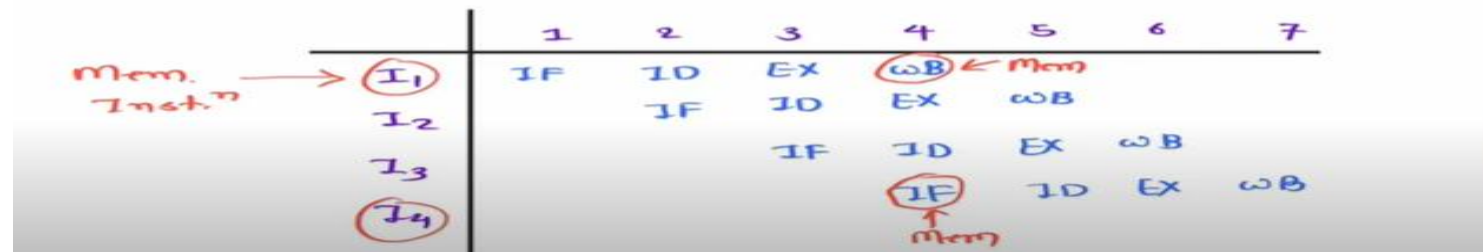
Clock cycle Stage	1	2	3	4	5	6	7	8	9	10	•	•	•	59	60	61	62	63
1	1	2	3	4	5	6	7	2	3	4	•	•	•	5	6	7	8	9
2	-	1	2	3	4	5	6	7	2	3	•	•	•	4	5	6	7	8
3	-	-	1	2	3	4	5	6	7	2	•	•	•	3	4	5	6	7
							↑	↑								↑	↑	
							branch taken	annulled						branch not taken		not annulled		

Structural Hazards

A structural hazard, also called a resource conflict, occurs when two or more instructions require access to the same hardware resource simultaneously, and the hardware cannot support the required parallel access.

→ Lets take four stage pipeline :

1. Instruction Fetch (IF)
2. Instruction decode + Operand decode (ID)
3. Execute Instruction (EX)
4. Writeback (WB)



To minimize such stalls, one common technique is Resource Duplication or Memory Split, where: Instruction Memory (Code Memory) stores program instructions.

Data Memory stores program data.

This separation allows simultaneous access—one stage can fetch instructions while another accesses data—thus avoiding memory-related structural dependencies.

RISC Pipeline

- Think about RISC Processor:
- Lets Assume (for use of efficient instruction pipeline):
- To implement an instruction pipeline we are using a small number of Sub-operations, with each being **executed in one clock cycle**.
- And we have **a fixed-length instruction format**, the decoding of the operation can occur at the same time as the register selection.
- Requires more lines of code and bigger RAM space.
- The instruction pipeline **can be implemented with two or three segments**.
 - One segment fetches the instruction from program memory [here denoted by I]
 - The other segment executes the instruction in the ALU [here denoted by A]
 - instruction decoding, data manipulation, effective (or branching) address calculation
 - Third segment may be used to store the result of the ALU operation in a destination register [here represented as E]

RISC Pipeline

- There are generally three types of instructions:
 - **The data transfer instructions**✓✓
 - The data manipulation instructions
 - The program control instructions
- The data transfer instructions in RISC are limited to **load** and **store** instructions.
- These instructions generally use register indirect addressing and there is a chance of conflict.
 - Few designs of RISC processors use two separate buses with two memories

RISC Pipeline

- In RISC we use two instructions for transfer: LOAD & STORE
 - LOAD transfers memory word to a register (Accumulator)
 - STORE transfers the contents of register to a memory
- There are two cases:
 - Data Conflicts (Solved via Delayed Load)
 - Branch Conflicts (Solved via Delayed Branch)
- Typical Example:
 - 1. LOAD: $R1 \leftarrow M[\text{address } 1]$
 - 2. LOAD: $R2 \leftarrow M[\text{address } 2]$
 - 3. ADD: $R3 \leftarrow R1 + R2$
 - 4. STORE: $M[\text{address } 3] \leftarrow R3$
- There will be a data conflict in instruction 3 above because operand in R2 is not available yet in the A segment.

RISC Pipeline

- The E segment in clock cycle 4 is in a process of placing the memory data into R2.
- The A segment in clock cycle 4 is using the data from R2.
- A **no-op** instruction is used to advance one clock cycle in order to compensate for the data conflict in the pipeline.
- The advantage of the delayed load approach is that the data dependency is taken care of by the compiler rather than the hardware.

Clock cycles:	1	2	3	4	5	6
1. Load R1	I	A	E			
2. Load R2		I	A	E		
3. Add R1+R2			I	A	E	
4. Store R3				I	A	E

Clock cycle:	1	2	3	4	5	6	7
1. Load R1	I	A	E				
2. Load R2		I	A	E			
3. No-operation			I	A	E		
4. Add R1+R2				I	A	E	
5. Store R3					I	A	E

RISC Pipeline

- Delayed Branch:
- Example:
 - Load from memory to R1
 - Increment R2
 - Add R3 to R4
 - Subtract R5 from R6
 - Branch to address X
- The compiler is designed to analyze the instructions before and after the branch and rearrange the program sequence by inserting useful instructions in the delay steps.
- It is up to the compiler to find useful instructions to put after the branch instruction.
- **Failing** that, the compiler can insert no-op instructions.

Delayed branching is a control-hazard handling technique in which the instruction immediately after a branch is always executed, regardless of whether the branch is taken or not.

In a pipeline:

Branch decision is known late

Next instruction is already fetched

—

RISC Pipeline

Clock cycles:	1	2	3	4	5	6	7	8	9	10
1. Load	I	A	E							
2. Increment		I	A	E						
3. Add			I	A	E					
4. Subtract				I	A	E				
5. Branch to X					I	A	E			
6. No-operation						I	A	E		
7. No-operation							I	A	E	
8. Instruction in X								I	A	E

Clock cycles:	1	2	3	4	5	6	7	8
1. Load	I	A	E					
2. Increment		I	A	E				
3. Branch to X			I	A	E			
4. Add				I	A	E		
5. Subtract					I	A	E	
6. Instruction in X						I	A	E

Vector Processing

- Vector:

Beyond Conventional Computing

- There is a class of computational problems that are beyond the capabilities of a conventional computer. These problems require a vast number of computations that will take a conventional computer days or even weeks to complete.

Vector Processing Applications

- Problems that can be efficiently formulated in terms of vectors and matrices:
 - Long-range weather forecasting
 - Seismic data analysis
 - Aerodynamics and space flight simulations
 - Artificial intelligence and expert systems
 - Mapping the human genome
 - Image processing

Vector Processor (Computer)

- Ability to process vectors and matrices much faster than conventional computers.

Fortran Language

```
DO 20 I = 1, 100
```

```
C(I) = B(I) + A(I)
```

Conventional Computer (Machine Language)

```
20 Initialize I = 0
```

```
Read A(I)
```

```
Read B(I)
```

```
Store C(I) = A(I) + B(I)
```

```
Increment I = I + 1
```

```
If I ≤ 100 goto 20
```

Vector Computer

```
C(1:100) = A(1:100) + B(1:100)
```

Without vector processing (scalar):
for i in range(4):

$C[i] = A[i] + B[i]$

 # Does: $C[0]=A[0]+B[0]$, WAIT

 # Then: $C[1]=A[1]+B[1]$, WAIT

 # Then: $C[2]=A[2]+B[2]$, WAIT

 # Then: $C[3]=A[3]+B[3]$

 # 4 separate operations

With vector processing:

$C = A + B \leftarrow$ ONE operation adds

ALL elements

$A = [1, 2, 3, 4]$

$B = [5, 6, 7, 8]$

$C = [6, 8, 10, 12] \leftarrow$ All 4 additions done simultaneously!

Vector is an ordered collection of scalar elements of the same data type stored in contiguous memory locations.

Each element is a scalar (int, float, double, etc.)

All elements are homogeneous

Accessed using an index

Treated as a single entity by vector instructions

Scalar Processing

One instruction operates on one data element

Repeated execution needed for arrays

Vector processors follow the SIMD (Single Instruction Multiple Data) model

Single control unit

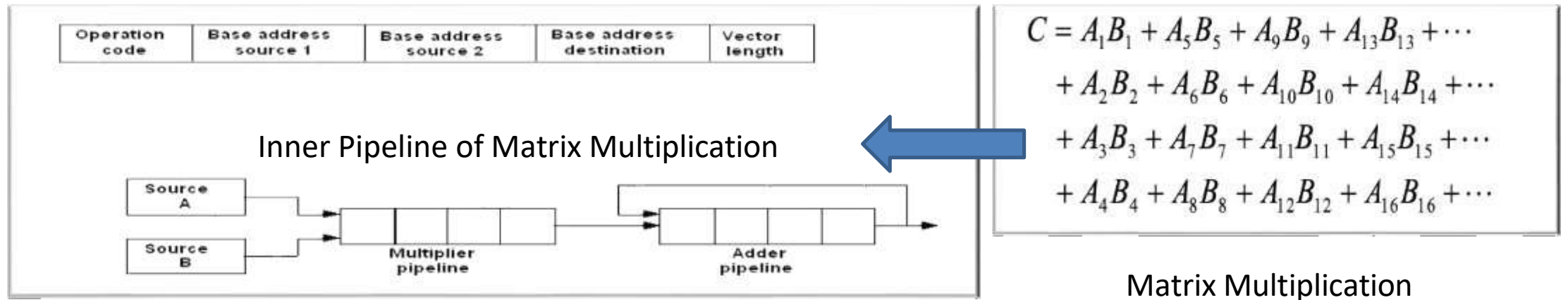
One instruction stream

Multiple data elements processed simultaneously

Lockstep execution

Vector Processing

- Vector Instruction Format:



- Vector Instruction (Operation) Types (SIX Types):

- Vector-Vector Instruction
- Vector-Scalar Instruction
- Vector Memory Instruction
- Vector Reduction Instruction
- Gather and Scatter Instruction
- Masking Instruction

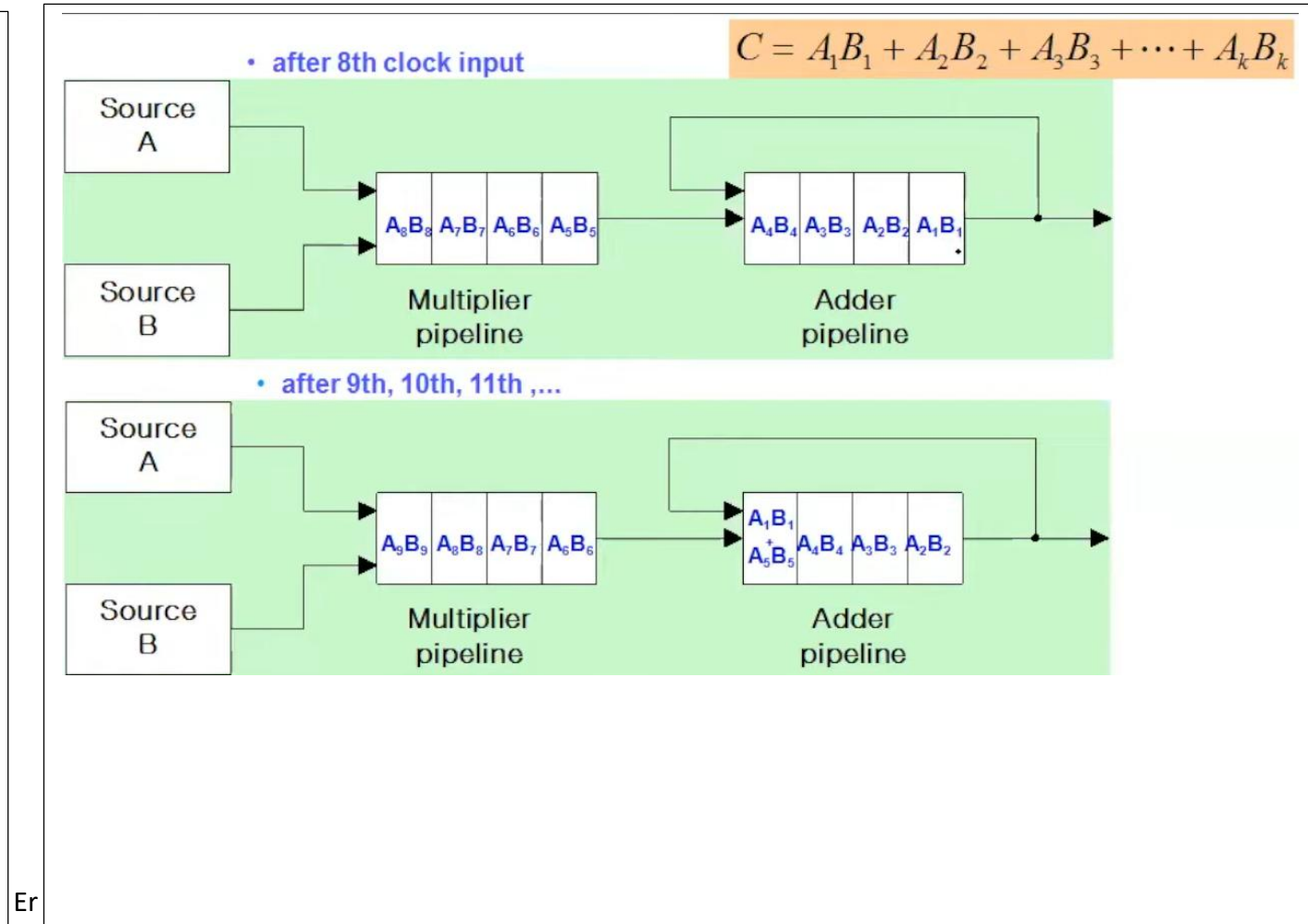
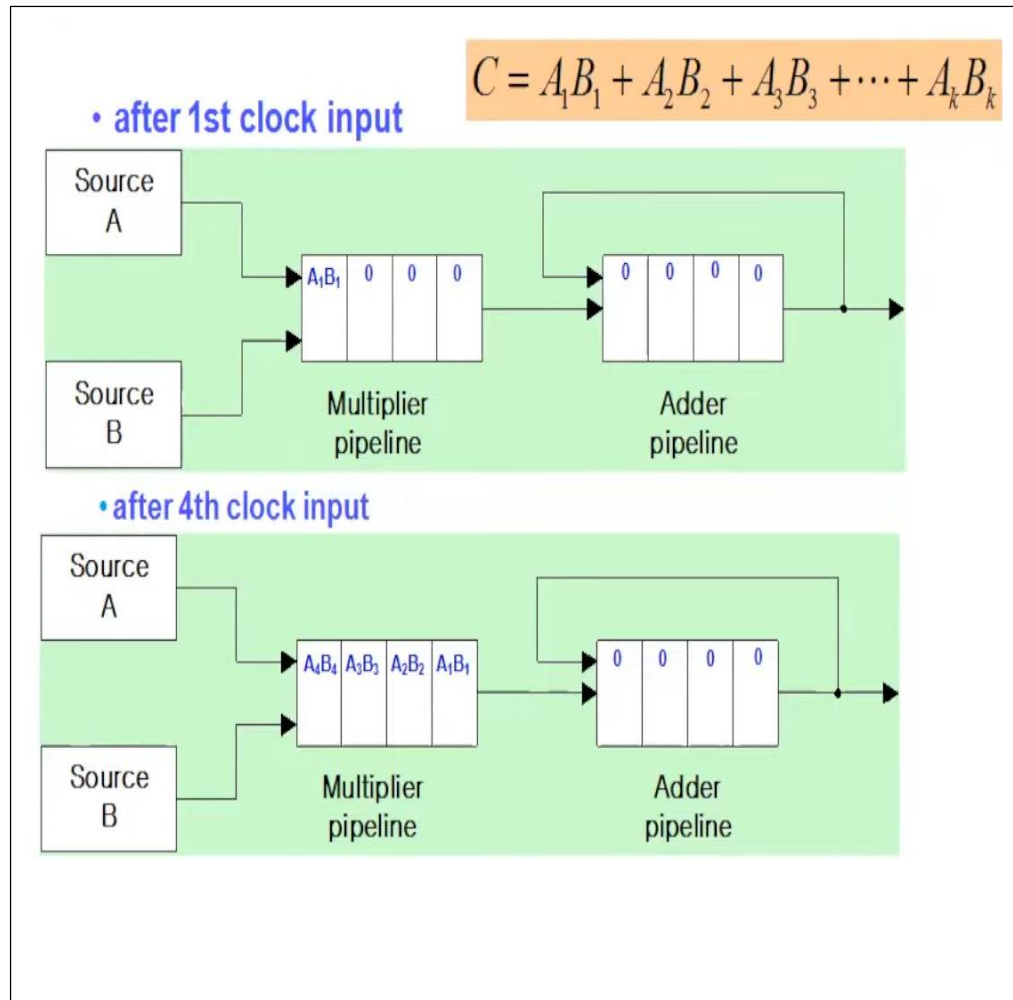
$$c_{11} = a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

Floating point multiplier pipeline with 4 segments

Floating point adder pipeline with 4 segments

This indicates that the inner product calculation is broken down into stages (pipelined) across two specialized units—one for multiplication and one for addition—each consisting of 4 pipeline stages to improve computational throughput.



1. Setup:

- Data (A_i and B_i) come from memory or registers.
- There are two separate pipelines:
 - A 4-stage **floating-point multiplier** (calculates $A_i \times B_i$).
 - A 4-stage **floating-point adder** (adds products to a running sum).

2. Initial delay (latency):

- For the first 8 cycles, the adder outputs **zero** because both pipelines are still filling.
- Each cycle, one new pair (A_i , B_i) enters the multiplier pipeline.

3. Pipeline filling process:

- After 4 cycles, the **first product** emerges from the multiplier and enters the adder pipeline.
- For the next 4 cycles, the adder is still filling, so it adds products to **zero**.
- At the end of cycle 8:
 - Multiplier pipeline contains $A_5B_5 \dots A_8B_8$.
 - Adder pipeline contains $A_1B_1 \dots A_4B_4$ (being summed gradually).

4. Steady state (from cycle 9 onward):

- Both pipelines are full.
- Each cycle:
 - A new product emerges from the multiplier.
 - The previous partial sum emerges from the adder and is added to the new product (feedback into adder input).
- The total sum is broken into **four overlapping partial sums** handled concurrently in the adder's four segments.

This pipelining speeds up vector operations like dot products by parallelizing multiplication and addition in stages, despite initial latency.

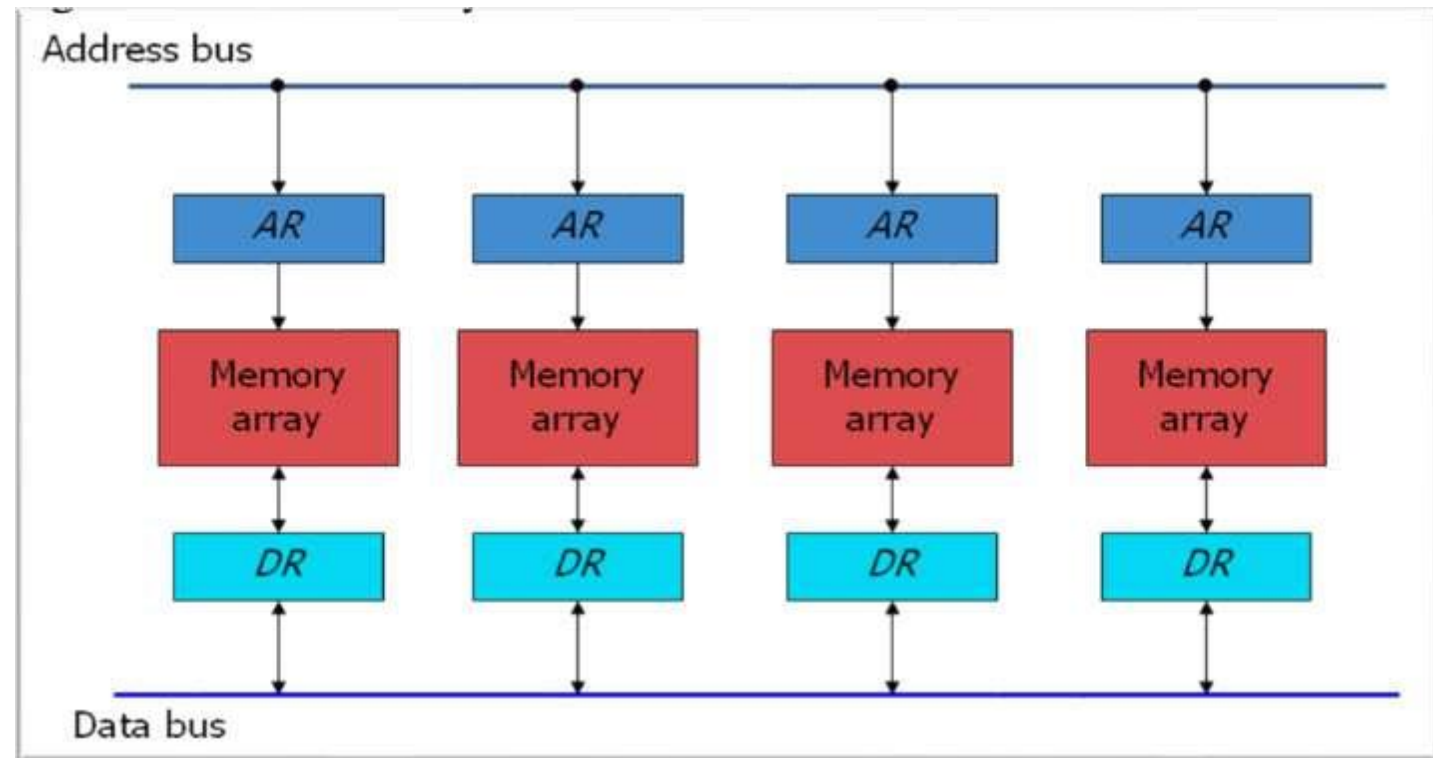
$$\begin{aligned} \bullet \quad C = & A_1 B_1 + A_5 B_5 + A_9 B_9 + A_{13} B_{13} + \dots \\ & + A_2 B_2 + A_6 B_6 + A_{10} B_{10} + A_{14} B_{14} + \dots \\ & + A_3 B_3 + A_7 B_7 + A_{11} B_{11} + A_{15} B_{15} + \dots \\ & + A_4 B_4 + A_8 B_8 + A_{12} B_{12} + A_{16} B_{16} + \dots \end{aligned}$$

Vector Processing

- **Memory Interleaving**
- Pipeline and vector processors often require simultaneous access to memory from two or more sources.
- For Example:
 - An instruction pipeline may require the fetching of an instruction and an operand at the same time from two different segments.
 - An arithmetic pipeline usually requires two or more operands to enter the pipeline at the same time.
- Instead of using two memory buses for simultaneous access, the memory can be partitioned into a number of modules connected to a common memory address and data buses.
- A memory module is a memory array together with its own address and data registers.

Vector Processing

- The advantage of a modular memory is that it allows the use of a technique called interleaving.
- In an interleaved memory, different sets of addresses are assigned to different memory modules.



Vector Processing

- A commercial computer with vector instructions and pipelined floating-point arithmetic operations is referred to as a supercomputer.
- This is augmented by instructions that process vectors and combinations of scalars and vectors.
- A supercomputer is a computer system best known for its high computational speed, fast and large memory systems, and the extensive use of parallel processing.
- It is equipped with multiple functional units and each unit has its own pipeline configuration.
- It is specifically optimized for the type of numerical calculations involving vectors and matrices of floating-point numbers.
- They are limited in their use to a number of scientific applications, such as numerical weather forecasting, seismic wave analysis, and space research.
- Note:
 - Floating-point operations per second is referred to as **flops**.
 - A typical supercomputer has a basic cycle time of 4 to 20 ns.

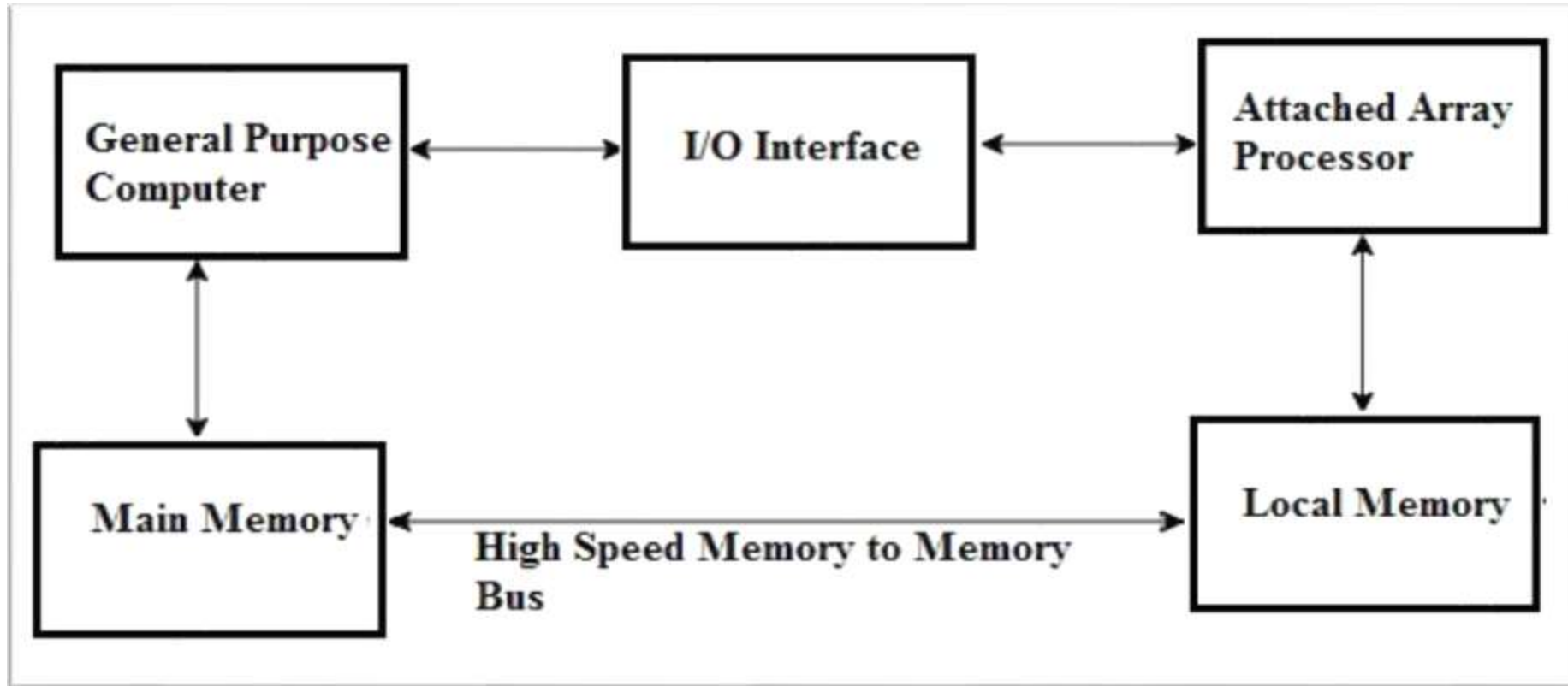
Array Processing

- An array processor is a processor that performs computations on large arrays of data.
- The term is used to refer to TWO different types of processors.
- Attached array processor:
 - an auxiliary processor (considered as peripheral device).
 - intended to improve the performance of the host computer in specific numerical computation tasks.
- SIMD array processor:
 - has a single-instruction multiple-data organization.
 - It manipulates vector instructions by means of multiple functional units responding to a common instruction.

Array Processing

- **Attached Array Processor:**
- Its purpose is to enhance the performance of the computer by providing vector processing for complex scientific applications.
 - Parallel processing with multiple functional units
- It is generally interconnected to a host computer.
- For example, when attached to a VAX (virtual address extension) 11 computer, the FSP-164/MAX from Floating-Point Systems increases the computing power of the VAX to 100 megaflops.
- The objective of the attached array processor is to provide vector manipulation capabilities to a conventional computer. [figure next slide]

Array Processing

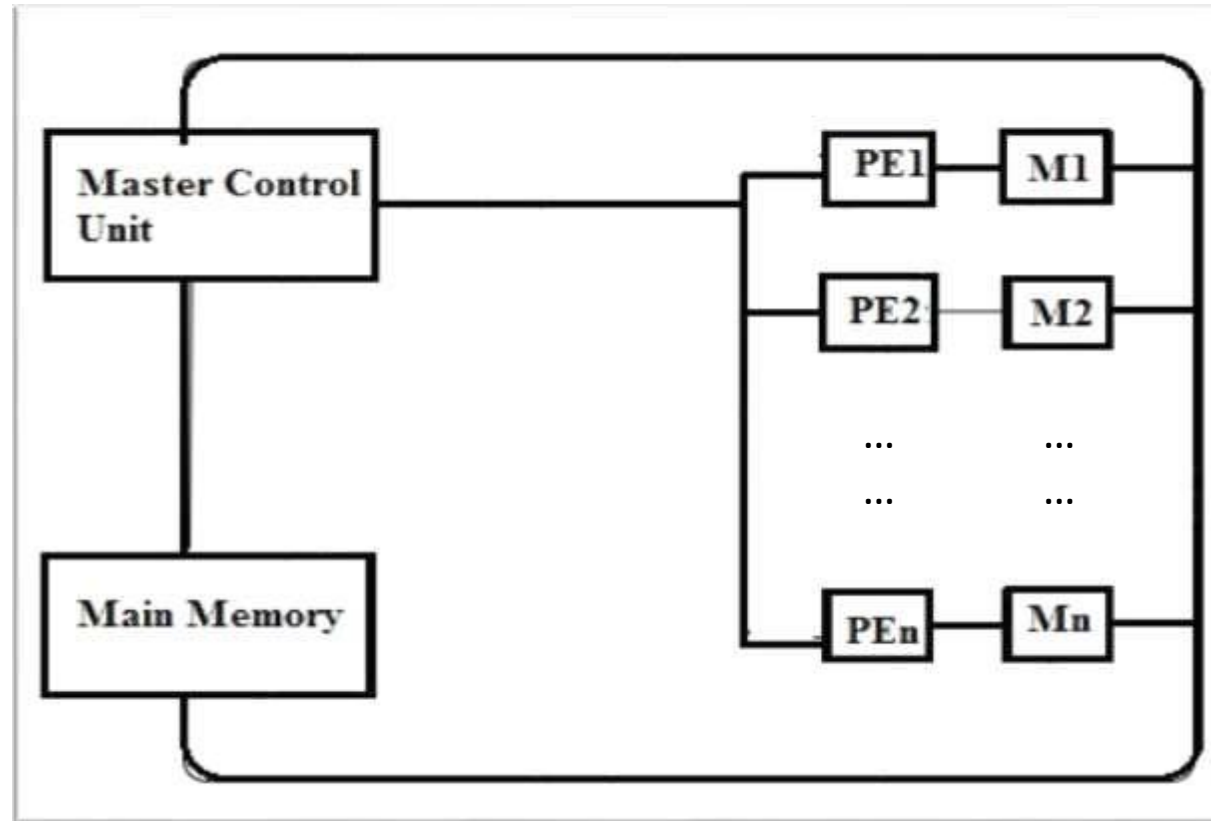


Attached Array Processor

Array Processing

- **SIMD Array Processor:**
- A SIMD array processor is a computer with multiple processing units operating in parallel.
- It contains a set of identical processing elements (PEs), each having a local memory M.
- Each PE includes an ALU, a floating-point arithmetic unit, and working registers.
- Vector instructions are broadcast to all PEs simultaneously.
- Masking schemes are used to control the status of each PE during the execution of vector instructions.
- Each PE has a flag that is set when the PE is active and reset when the PE is inactive.
- For example, the ILLIAC (**Illinoi Automatic Computer**) IV computer developed at the University of Illinois and manufactured by the Burroughs Corp. (**Figure Next Slide**)

Array Processing



SIMD Array Processor